

Exercise 2: Performance with @defer Blocks

Context: When you wrap a component in a `@defer` block, the Angular compiler separates that component's code into a distinct JavaScript chunk file during the compilation build. The main application JavaScript bundle loads instantly without the deferred chunk. The chunk only downloads when the specific trigger condition fires. This is not a CSS `display: none` layout mask; the source code literally does not exist inside the browser's memory until the trigger fires.

Step 1: Generate the Components

You need two new components: the instructor dashboard (the parent page) and the analytics chart (the heavy child component that gets deferred):

```
ng generate component features/instructor-dashboard
ng generate component ui/analytics-chart
```

[!NOTE]

Step 2: Build the Analytics Chart Component

This component simulates a heavyweight charting library. In a production TMS, this would be a real chart framework (such as Chart.js or ngx-charts) rendering enrollment trends. For this exercise, a styled placeholder with enough internal logic is sufficient to produce a measurable separate code chunk.

Open `src/app/ui/analytics-chart/analytics-chart.component.ts` and replace its contents:

```
import { Component, computed, input } from '@angular/core';
import { Enrollment } from '../../models/enrollment.model';

@Component({
  selector: 'tms-analytics-chart',
  standalone: true,
  template: `
    <div class="chart-container">
      <h3>Enrollment Analytics</h3>
      <div class="chart-bars">
        <div class="bar approved"
[style.height.px]="approvedHeight()">
          <span>Approved</span>
        </div>
        <div class="bar pending"
[style.height.px]="pendingHeight()">
          <span>Pending</span>
        </div>
        <div class="bar rejected"
[style.height.px]="rejectedHeight()">
```

```

        <span>Rejected</span>
      </div>
    </div>
    <p class="chart-summary">
      Total records: {{ data().length }}
    </p>
  </div>
  `
  ,
  styleUrls: ['./analytics-chart.component.scss']
})
export class AnalyticsChartComponent {
  data = input.required<Enrollment[]>();

  // computed() memoizes the result – the filter only re-runs when
  // data() changes,
  // not on every change detection cycle. This is the signal-first
  // pattern M9 teaches.
  approvedHeight = computed(() => {
    const count = this.data().filter(e => e.status ===
'Approved').length;
    return Math.max(20, count * 3);
  });

  pendingHeight = computed(() => {
    const count = this.data().filter(e => e.status ===
'Pending').length;
    return Math.max(20, count * 3);
  });

  rejectedHeight = computed(() => {
    const count = this.data().filter(e => e.status ===
'Rejected').length;
    return Math.max(20, count * 3);
  });
}

```

[!NOTE]

Step 3: Build the Instructor Dashboard Parent Page

Open `src/app/features/instructor-dashboard/instructor-dashboard.component.ts` and replace its contents. The critical UI components (enrollment counts, pending approvals, and action buttons) render immediately, while the heavy chart defers:

```

import { Component, inject, OnInit } from '@angular/core';
import { EnrollmentStore } from '../../store/enrollment.store';
import { AnalyticsChartComponent } from '../../ui/analytics-chart/analytics-chart.component';

@Component({
  selector: 'tms-instructor-dashboard',

```

```

standalone: true,
imports: [AnalyticsChartComponent],
templateUrl: './instructor-dashboard.component.html',
styleUrl: './instructor-dashboard.component.scss'
})
export class InstructorDashboardComponent implements OnInit {
  store = inject(EnrollmentStore);

  ngOnInit() {
    this.store.loadEnrollments();
  }
}

```

Design Note: AnalyticsChartComponent remains declared in the standalone imports array so the compiler can validate the `<tms-analytics-chart>` selector in the template. Because the component is used exclusively inside a `@defer` block, Angular is smart enough to extract it into a separate chunk automatically without pulling it into the main bundle.

[!NOTE]

Step 4: Build the Dashboard Markup Template

Open `src/app/features/instructor-dashboard/instructor-dashboard.component.html` and append the following template layout containing your lazy-loading block rules:

```

<!-- Renders instantly on any connection speed -->
<div class="dashboard-header">
  <h1>Instructor Command Center</h1>
  <div class="kpi-row">
    <div class="kpi-card">
      <span class="kpi-value">{{ store.entities().length }}</span>
      <span class="kpi-label">Total Enrollments</span>
    </div>
    <div class="kpi-card pending">
      <span class="kpi-value">{{ store.pendingCount() }}</span>
      <span class="kpi-label">Pending Approval</span>
    </div>
  </div>
</div>

<!-- DEFERRED UI: The chart code lives in a separate .js chunk file -->
<div class="chart-section">
  @defer (on viewport; prefetch on idle(500)) {
    <tms-analytics-chart [data]="store.entities()" />
  } @placeholder {
    <div class="skeleton-chart">Scroll down to view analytics...
  </div>
  } @loading (minimum 500ms) {
    <div class="spinner">Downloading chart engine...</div>
  }
</div>

```

```
    } @error {  
      <p>Failed to load chart. Check your connection.</p>  
    }  
  </div>
```

[!NOTE]

Step 5: Wire the Dashboard into Your Application Routing

Open `src/app/app.routes.ts` and declare a lazy-loaded route mapping for the instructor dashboard view:

```
import { Routes } from '@angular/router';  
  
export const routes: Routes = [  
  {  
    path: 'dashboard',  
    loadChildren: () =>  
      import('./features/instructor-dashboard/instructor-  
        dashboard.component')  
        .then(m => m.InstructorDashboardComponent)  
  },  
  // ... your existing routes (enrollment-list, etc.)  
  { path: '', redirectTo: 'dashboard', pathMatch: 'full' }  
];
```

Lazy Loading Pipeline Architecture: The `loadComponent` syntax establishes lazy-loaded, route-level code splitting. Combined with the localized template `@defer` statement, you have built two distinct layers of code optimization: the route chunk handles the main dashboard framework, and the viewport trigger loads the auxiliary chart assets only when scrolled into view.

Architectural Deep Dive: What Each Piece Does

- **on viewport:** The chunk download triggers when the `<div class="chart-section">` scrolls into the browser's visible area. Internally, Angular leverages `IntersectionObserver`—the same native browser API that powers lazy-loaded image elements.
- **prefetch on idle(500):** Even before the user scrolls, Angular begins downloading the bundle chunk file during browser idle time. The `(500)` is a timeout constraint in milliseconds: if the client browser never reaches a true idle state (common on low-powered tablets), the prefetch fires after 500ms anyway. Without this timeout guard, `requestIdleCallback` can wait indefinitely on overloaded devices.
- **@placeholder:** Renders instantly on initial page load. This is what Liya sees while she reads enrollment counts—preventing a blank viewport screen. Enforcing a `min-height: 250px` layout style keeps it safely below the fold on mobile devices.
- **@loading (minimum 500ms):** Displays an active loading spinner component while the chunk downloads. The `minimum 500ms` parameter prevents an aggressive, jarring visual flash if the download network request completes in under 50ms.

- **@error**: Renders safely if the bundle chunk download fails completely (e.g., the client device drops offline mid-download).

A Note on OnPush Change Detection Optimization

Open your generated `instructor-dashboard.component.ts`. Notice that the Angular CLI configures every component class with a default execution property:

```
changeDetection: ChangeDetectionStrategy.OnPush
```

In older framework versions, Angular re-evaluated every component in the virtual DOM tree on any browser macro-task event—a legacy strategy now renamed Eager and deprecated. With **OnPush**, Angular only re-checks a component node when its signal-bound inputs change reference or when an internal `signal()` it reads emits a new value. Because every component in this module reads from the shared `EnrollmentStore`, **OnPush** integration works naturally. This optimization keeps your UI fast even when processing up to 5,000 table rows.

Exercise 2 Verification and Testing Matrix

Follow these verification steps in exact chronological order to confirm your asset-splitting compilation architecture:

1. **Build and Validate Chunk Splitting:** Execute a production build script from your terminal:

```
ng build
```

Examine the compilation directory summary logs. You should see a separate chunk file listed (e.g., `chunk-XXXX.js`). This confirms that the analytics chart source code lives inside its own discrete file, physically isolated from the main app bundle.

2. **Simulate a Slow Network Connection:** Initialize the local development server:

```
ng serve
```

Open Chrome Developer Tools (**F12**) and navigate to the **Network** tab. Set the throttling profile dropdown selector to **Slow 3G**, then reload `http://localhost:4200/dashboard`.

3. **Confirm Critical UI Load Priority:** The critical dashboard header ("Instructor Command Center"), total enrollment counts, and pending metrics must appear within 1–2 seconds. The deferred section must display your `@placeholder` skeleton layout ("Scroll down to view analytics...").
4. **Trigger Viewport Deferred Chunk Fetching:** Scroll down the viewport page until the chart element enters your view. Watch the active request streams inside your Network tab. A new

asynchronous JavaScript file entry (**chunk-XXXX.js**) must appear in the request list as the skeleton swaps to your placeholder spinner before rendering the chart.

5. **Validate Lazy Isolation Boundaries:** Scroll back to the top of the Network tab request log. Verify that the chart file asset was completely absent from the initial landing page load bundle requests, confirming it fetched exclusively when triggered by scrolling.